FlexQL Server Design Document

Rahul Das, 25CS60R15
https://github.com/rahul-das-713127/Flex_QL.git

April 7, 2026

Contents

1 Overview

FlexQL is a lightweight SQL-like database server with a TCP client protocol. The implementation in this repository is a C++17 codebase that focuses on:

- Persistent storage on disk (table data files + schema catalog, with optional WAL)
- Fast insert throughput (specialized fast paths, batching, and buffered IO)
- Basic relational querying (SELECT with single-condition WHERE; INNER JOIN)
- Performance accelerators (primary-key offset index; small-result caching)
- Multi-client concurrency (thread-per-connection)

This document describes the storage layout, indexing, caching, expiration handling, multithreading, and key design decisions.

2 Repository Structure

Project root (`flexql/`):

```
flexql/
  Makefile
  server_full.cpp         # Full server implementation (recommended)
  server.cpp              # Simpler/reference server (separate implementation)
  flexql.h                # C client API
  flexql_client.cpp       # Client library implementation
  client_repl.cpp         # REPL client
  benchmark_flexql.cpp    # Benchmark + unit tests client
  server                  # built artifact (make server)
  client                  # built artifact (make client)
  benchmark               # built artifact (make benchmark)
  data/
    catalog.txt           # schema catalog
    wal.log               # write-ahead log (when enabled)
    <TABLE>.data          # per-table data file(s)
```

3 Dependencies and Build Tooling

3.1 Language and Compiler

- C++17
- Typical toolchain: `g++` (or `clang++`)

3.2 System Libraries

- POSIX sockets and file APIs (Linux)
- `pthread` for threading (linked via `-lpthread`)

3.3 Build System

- `make` via the provided `Makefile`

4 Data Storage Design

4.1 Authoritative Storage is On Disk

The system does not use RAM as the primary data store. Persistent data resides in the `data/` directory.

- **Schemas**: persisted in `data/catalog.txt`
- **Table rows**: appended to per-table files `data/<TABLE>.data`
- **WAL (optional)**: appended to `data/wal.log` when enabled

RAM is used for speed (buffers, caching, indexes, and metadata), but disk is the source of truth.

4.2 Schema Catalog (`catalog.txt`)

Schemas are stored separately from table data. On startup, the server loads schemas from the catalog before serving queries.

Key properties:

- Schemas are stored under a canonical table name key (case-insensitive handling is implemented by normalizing names).
- Supported column types include: `INT`, `DECIMAL`, `VARCHAR(n)`, `DATETIME`.
- `EXPIRES_AT` is mandatory in every table schema.

4.3 Table Data Files (`<TABLE>.data`)

Each table has a dedicated file created under `data/`. Inserts are implemented as append-only writes.

4.3.1 Row Encoding

Rows are stored in a compact binary format designed for fast append and sequential scan:

- A header storing `ncols` (number of columns)
- For each column, a `uint32 length` followed by raw bytes of the value

Values are stored as their textual form (the same representation used by SQL parsing), which simplifies query execution and makes the storage format schema-agnostic.

4.3.2 Buffered Writes and Write Arenas

To reduce syscalls and improve throughput:

- Inserts serialize into an in-memory per-table *write arena* buffer.
- Once the arena reaches a threshold, it is flushed to the table `FILE*`.

- A large stdio buffer is configured using `setvbuf(..., _IOFBF, 8<<20)`.
- The server uses an async writer thread to flush arena buffers to disk without blocking the hot insert loop.

4.4 Database Recreation vs Persistence

On startup, the server may wipe `data/` depending on environment variables.

- The default and current behavior is to wipe the `data/` directory on every server start (fresh database per run).

This behavior ensures benchmarks and REPL sessions start from a clean database state unless the code is changed.

5 Write-Ahead Log (WAL)

5.1 Purpose

When enabled, WAL provides crash-recovery by logging SQL statements so that the server can replay them on startup.

Important scope:

- WAL improves durability and crash recovery.
- WAL does *not* provide replication, failover, or protection from disk loss.

5.2 Enable/Disable

- Default: WAL disabled.
- Enable: `FLEXQL_ENABLE_WAL=1`
- Force disable: `FLEXQL_DISABLE_WAL=1`

5.3 Record Format

The WAL file (`data/wal.log`) is a binary stream of records:

Field	Description
`uint32 len`	Number of bytes in the SQL statement
`len bytes sql`	SQL statement bytes (typically includes trailing semicolon)
`uint32 crc`	CRC32 of the SQL bytes

5.4 WAL Buffers and Flushing

To minimize lock contention and syscall overhead:

- Each client handler thread accumulates WAL records into a per-connection buffer.
- Periodically, the buffer is appended to a global WAL batch buffer under the global mutex.
- The batch is flushed when size/count thresholds are met, and also on disconnect.

5.5 Replay on Startup

On startup, the server replays `wal.log`:

- The server reads records sequentially.
- CRC is verified; on corruption/torn tail it stops and truncates the WAL to the last good record.
- Each SQL statement is executed via the normal execution engine.
- During replay, a flag disables WAL re-appending to prevent infinite log growth.

Operational note: For WAL replay to restore data, `data/` must be preserved across restarts (`FLEXQL_PRESERVE_DATA=1`). If the server wipes `data/` at startup, there is nothing to replay.

6 SQL Layer and Schema Enforcement

6.1 CREATE TABLE

CREATE TABLE parses and stores the schema. The server enforces:

- Only supported types are allowed: INT, DECIMAL, VARCHAR(n), DATETIME.
- VARCHAR requires `n > 0`.
- `EXPIRES_AT` must exist.

6.2 INSERT

INSERT supports inserting rows into a table. The implementation enforces:

- Column count matches the schema.
- Each column value matches its declared type.
- VARCHAR length is limited to `n`.
- `EXPIRES_AT` must be a valid positive integer timestamp.

Validation is applied for both:

- Normal insert path
- Fast insert path
- Ultra-fast bulk insert path

The server supports **multi-row inserts** of the form: `INSERT INTO T VALUES (..),(..),(..);` which allows client-side batching to reduce round trips.

6.3 SELECT

SELECT supports:

- `SELECT * FROM T;`
- `SELECT C1, C2 FROM T;`
- Optional single-condition WHERE clause

6.4 WHERE and JOIN Operators

The condition operator set supported in WHERE and JOIN includes:

- `=, >, <, >=, <=`

INNER JOIN is supported with an ON condition and an optional WHERE clause.

7 Expiration Timestamp Handling

7.1 EXPIRES_AT Column

Every table must contain an `EXPIRES_AT` column. Inserts require a valid expiration value.

7.2 Query-Time Filtering

During scans (including SELECT and JOIN), expired rows are filtered out. In other words:

- Rows with `EXPIRES_AT` in the past are skipped.
- Expiration enforcement happens at read time.

7.3 Design Decision

The server does not run a background vacuum to delete expired rows from disk. Instead:

- Reads ignore expired rows.
- Storage usage can grow unless compaction/vacuum is added.

8 Indexing

8.1 Primary Key Offset Index

To accelerate equality lookups on a likely primary key column:

- The server detects a "probable PK" column (typically the first column or a column named `ID`).
- If it is INT, the server enables an offset index: `pk_offsets`.
- The index maps PK value $\rightarrow$ file offset for fast retrieval.

8.2 Index Maintenance

- Inserts update index state (or mark it invalid for deferred rebuild depending on the path).
- On startup (after loading schemas), the server can rebuild indexes by scanning table files.

8.3 Index Usage in Queries

If a WHERE clause matches the PK column (or qualified form), the query executor can take a fast path to avoid full-table scans.

9 Caching Strategy

9.1 Goal

Reduce latency for repeated SELECT queries returning small result sets.

9.2 Mechanism

- An in-memory cache stores results keyed by the SQL string.
- Cache is an LRU-like structure with bounded capacity.
- Cache entries are used only for safe/eligible SELECTs (small results).

9.3 Invalidation

Writes invalidate cached results:

- Inserts set a global dirty flag (atomic) or directly clear cache depending on the code path.

10 Multithreading and Concurrency

10.1 Threading Model

- The server listens on TCP port 9000.
- Each accepted connection is handled by a dedicated `std::thread` running `handle_client`.

10.2 Synchronization

- **Global mutex** protects shared metadata (schemas, runtime maps, WAL batch buffer).
- **Per-table spinlock** is used in the hot insert path to reduce overhead under heavy insert loads.

10.3 Write Path Concurrency

- For repeated inserts into the same table, the handler can hold the per-table spinlock across multiple rows received in a burst.
- The write arena reduces file IO contention and amortizes syscall overhead.

10.4 Read Path Concurrency

- SELECT execution may require flushing pending write arenas before reads for visibility.
- This is coordinated via flush hooks and the global mutex.

11 Operational Guide

11.1 Environment Variables

Variable	Meaning

`FLEXQL_PRESERVE_DATA=1`	Preserve `data/` across server restarts (otherwise wiped on start).
`FLEXQL_ENABLE_WAL=1`	Enable WAL logging to `data/wal.log`.
`FLEXQL_DISABLE_WAL=1`	Force WAL disabled.
`FLEXQL_PIPELINE_INSERTS=1`	Client-side optimization: pipeline inserts.
`FLEXQL_BULK_ACK_INSERTS=1`	Client/server optimization: bulk acknowledgements.

11.2 Stopping a Server Running in Background

If the server is running in the background and you see errors like `bind failed`, find and kill the process:

```
ss -ltnp | grep ":9000"
# identify PID, then:
kill <PID>
# if it won't stop:
kill -9 <PID>



lsof -iTCP:9000 -sTCP:LISTEN -n -P
kill <PID>
```

12 Known Limitations and Future Improvements

Using lsof

- Expired rows are filtered at read time; there is no vacuum/compaction.
- WAL replay is SQL-replay based; it re-executes statements rather than replaying page-level changes.
- Indexing is currently focused on a probable INT primary key column.
- Query planner is minimal (single WHERE condition; no AND/OR).